```
# ====================================================
# EXPERIMENT 1
# LINEAR REGRESSION USING GRADIENT DESCENT
# AND NORMAL EQUATION
# ====================================================

# ====================================================
# SECTION 1: IMPORT REQUIRED LIBRARIES
# Study: NumPy, Pandas, Matplotlib and sklearn
# ====================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# ====================================================
# SECTION 2: LOAD THE DATASET
# Study: Loading California Housing Dataset
# and converting it into a DataFrame
# ====================================================

housing = fetch_california_housing()

data = pd.DataFrame(housing.data, columns=housing.feature_names)
data["Price"] = housing.target
```

```
# ====================================================
# SECTION 3: SELECT INPUT AND OUTPUT
# Study: x is input (AveRooms)
# y is output (Price)
# ====================================================

x = data[['AveRooms']].values
y = data['Price'].values

# ====================================================
# SECTION 4: SPLIT DATA INTO TRAINING AND TESTING
# Study: 80% training and 20% testing
# ====================================================

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)

# ====================================================
# SECTION 5: FEATURE SCALING
# Study: StandardScaler standardizes input values
# ====================================================

scaler = StandardScaler()

x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

```
# ====================================================
# SECTION 6: GRADIENT DESCENT INITIALIZATION
# Study: Initialize weight, bias, learning rate,
# epochs, cost history and number of samples
# ====================================================

# ====================================================
# Gradient Descent
# ====================================================

w = 0
b = 0
learning_rate = 0.01
epochs = 1000

cost_history = []

n = len(x_train_scaled)

# ====================================================
# SECTION 7: GRADIENT DESCENT TRAINING
# Study: Predict values, calculate gradients,
# update weight and bias, calculate cost
# ====================================================
for i in range(epochs):

        y_pred = w * x_train_scaled.flatten() + b

        dw = (1 / n) * np.sum(
           (y_pred - y_train) * x_train_scaled.flatten()
        )
```

```
        db = (1 / n) * np.sum(y_pred - y_train)

        w = w - learning_rate * dw
        b = b - learning_rate * db

        cost = (1 / (2 * n)) * np.sum(
            (y_pred - y_train) ** 2
        )

        cost_history.append(cost)

        if i % 100 == 0:
            print(f"Epoch {i}, Cost = {cost:.4f}")

# ======================================================
# SECTION 8: GRADIENT DESCENT PREDICTION
# AND PERFORMANCE EVALUATION
# Study: Predict test data, print weight, bias,
# MSE and R2 score
# ======================================================

y_pred_gd = w * x_test_scaled.flatten() + b

print("Gradient Descent")
print("----------------")
print("Weight: ", w)
print("Bias: ", b)
print("MSE:", mean_squared_error(y_test, y_pred_gd))
print("R2 Score:", r2_score(y_test, y_pred_gd))
```

```
# ====================================================
# SECTION 9: NORMAL EQUATION
# Study: Calculate weight and bias directly
# using mean values without iterations
# ====================================================

# ====================================================
# Normal Equation - Easy Version
# ====================================================

x_mean = np.mean(x_train)
y_mean = np.mean(y_train)

w_ne = np.sum(
    (x_train.flatten() - x_mean) *
    (y_train - y_mean)
) / np.sum(
    (x_train.flatten() - x_mean) ** 2
)

b_ne = y_mean - w_ne * x_mean
```

```
# ====================================================
# SECTION 10: NORMAL EQUATION PREDICTION
# AND PERFORMANCE EVALUATION
# Study: Predict house prices and calculate
# MSE and R2 score
# ====================================================

y_pred_ne = w_ne * x_test.flatten() + b_ne

print("\nNormal Equation")
print("----------------")
print("Weight: ", w_ne)
print("Bias: ", b_ne)
print("MSE:", mean_squared_error(y_test, y_pred_ne))
print("R2 Score:", r2_score(y_test, y_pred_ne))

# ====================================================
# SECTION 11: SORT VALUES FOR PLOTTING
# Study: Arrange test data in ascending order
# so the regression line is drawn correctly
# ====================================================
# ====================================================
# Sort values for plotting
# ====================================================

idx = np.argsort(x_test.flatten())
x_sorted = x_test.flatten()[idx]
y_test_sorted = y_test[idx]

y_pred_gd_sorted = y_pred_gd[idx]
y_pred_ne_sorted = y_pred_ne[idx
```

```
# ======================================================
# SECTION 12: CREATE FIGURE FOR GRAPHS
# Study: Create one figure with two subplots
# ======================================================



# ======================================================
# Plotting
# ======================================================

plt.figure(figsize=(12, 5))

# ======================================================
# SECTION 13: GRADIENT DESCENT COST CURVE
# Study: Plot cost history against iterations
# A decreasing curve indicates convergence
# ======================================================

# Gradient Descent Cost Curve
plt.subplot(1, 2, 1)

plt.plot(
    cost_history,
    color='navy',
    linewidth=2
)

plt.title('Gradient descent cost convergence curve')
plt.grid(True, linestyle='--', alpha=0.6)
```

```
# ====================================================
# SECTION 14: LINEAR REGRESSION GRAPH
# Study: Plot actual house prices and the
# predicted line from Normal Equation
# ====================================================
# Regression Plot
plt.subplot(1, 2, 2)

plt.scatter(
    x_test,
    y_test,
    color='blue',
    alpha=0.5,
    label='Actual Data'
)

plt.plot(
    x_sorted,
    y_pred_ne_sorted,
    color='green',
    linewidth=2,
    label='Normal Equation'
)

plt.xlabel("Average Rooms")
plt.ylabel("House Price")
plt.title("Normal Equation Regression")
plt.legend()

plt.tight_layout()
plt.show()
```

```
# ====================================================
# EXPERIMENT 2
# LINEAR AND POLYNOMIAL REGRESSION
# USING AUTO MPG DATASET
# ====================================================



# ====================================================
# SECTION 1: IMPORT REQUIRED LIBRARIES
# Study: NumPy, Pandas, Matplotlib, sklearn
# ====================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score

# ====================================================
# SECTION 2: LOAD THE AUTO MPG DATASET
# Study: Read CSV file using Pandas
# ====================================================

url="https://raw.githubusercontent.com/mwaskom/seaborn-data/master/mpg.csv"
df=pd.read_csv(url)
df.head()
```

```
# ====================================================
# SECTION 3: SELECT INPUT AND OUTPUT FEATURES
# Study: x = displacement, y = mpg
# Check missing values
# ====================================================

x=df[['displacement']]
y=df['mpg']
print("missing values in ",x.isnull().sum())
print("Missing values in mpg:",y.isnull().sum())



# ====================================================
# SECTION 4: REMOVE MISSING VALUES
# Study: dropna() removes rows with missing values
# Display the first five values of x and y
# ====================================================

data=df[['displacement','mpg']].dropna()
x=data[['displacement']]
y=data['mpg']
print("Feature:")
print(x.head())
print("\nTarget:")
print(y.head())
```

```
# ======================================================
# SECTION 5: SCATTER PLOT OF DATASET
# Study: Visualize displacement vs MPG
# ======================================================

plt.figure(figsize=(8,5))
plt.scatter(x,y)
plt.xlabel("Engine Displacement")
plt.ylabel("Miles Per Gallon (MPG)")
plt.grid(True)
plt.show()



# ======================================================
# SECTION 6: SPLIT DATA INTO TRAINING AND TESTING
# Study: 80% training and 20% testing
# ======================================================

x_train,x_test,y_train,y_test=train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42
)
print("Training samples:",len(x_train))
print("Testing samples:",len(x_test))
```

```
# ====================================================
# SECTION 7: TRAIN LINEAR REGRESSION MODEL
# Study: fit() trains the model
# predict() predicts MPG
# MSE and R2 evaluate performance
# ====================================================

linear_model = LinearRegression()
linear_model.fit(x_train, y_train)
y_pred_linear = linear_model.predict(x_test)
mse_linear = mean_squared_error(y_test, y_pred_linear)
r2_linear = r2_score(y_test, y_pred_linear)
print("Linear Regression Results")
print("------------------------")
print("MSE:", mse_linear)
print("R-squared:", r2_linear)

# ====================================================
# SECTION 8: STORE LINEAR REGRESSION RESULTS
# Study: Store model, degree, MSE and R2
# in a list for comparison
# ====================================================

results = []
polynomial_models = {}
results.append({
    'Model': 'Linear Regression',
    'Degree': 1,
    'MSE': mse_linear,
    'R-squared': r2_linear
})
```

```
# ====================================================
# SECTION 9: POLYNOMIAL REGRESSION
# Study: Train models of degrees 2, 3, 4 and 5
# PolynomialFeatures creates polynomial inputs
# ====================================================

for degree in range(2, 6):
    poly = PolynomialFeatures(degree=degree)
    x_train_poly = poly.fit_transform(x_train)
    x_test_poly = poly.transform(x_test)

    poly_model = LinearRegression()
    poly_model.fit(x_train_poly, y_train)
    y_pred_poly = poly_model.predict(x_test_poly)
    mse = mean_squared_error(y_test, y_pred_poly)
    r2 = r2_score(y_test, y_pred_poly)
    results.append({
        'Model': 'Polynomial Regression',
        'Degree': degree,
        'MSE': mse,
        'R-squared': r2
    })
    polynomial_models[degree] = {
        'poly': poly,
        'model': poly_model
    }
```

```
# ====================================================
# SECTION 10: DISPLAY MODEL COMPARISON TABLE
# Study: Convert results list into DataFrame
# Compare MSE and R2 of all models
# ====================================================

results_df = pd.DataFrame(results)
results_df



# ====================================================
# SECTION 11: PREPARE VALUES FOR REGRESSION PLOT
# Study: Generate 300 displacement values
# for drawing smooth regression curves
# ====================================================

x_plot = pd.DataFrame({
    'displacement': np.linspace(
        x['displacement'].min(),
        x['displacement'].max(),
        300)
})



# ====================================================
# SECTION 12: PLOT LINEAR AND POLYNOMIAL REGRESSION
# Study: Scatter plot of actual data
# Plot linear regression line
# Plot polynomial curves of degrees 2 to 5
# ====================================================
```

```
plt.figure(figsize=(10, 6))

plt.scatter(
    x['displacement'],
    y,
    label='Actual Data'
)

y_plot_linear = linear_model.predict(x_plot)
plt.plot(
    x_plot['displacement'],
    y_plot_linear,
    label='Linear Regression'
)

for degree in range(2, 6):
    poly = polynomial_models[degree]['poly']
    poly_model = polynomial_models[degree]['model']
    x_plot_poly = poly.transform(x_plot)
    y_plot_poly = poly_model.predict(x_plot_poly)
    plt.plot(
        x_plot['displacement'],
        y_plot_poly,
        label=f'Polynomial Degree {degree}'
    )

plt.xlabel('Engine Displacement')
plt.ylabel('Miles Per Gallon (MPG)')
plt.title(
    'Linear and Polynomial Regression: Displacement vs MPG'
)
```

```
plt.legend()
plt.grid(True)
plt.show()



# ====================================================
# SECTION 13: MSE COMPARISON BAR GRAPH
# Study: Compare Mean Squared Error of models
# Lower MSE indicates better performance
# ====================================================

import matplotlib.pyplot as plt
plt.figure(figsize=(8, 5))
bars = plt.bar(
    results_df['Degree'].astype(str),
    results_df['MSE']
)
plt.bar_label(bars, fmt='%.2f', padding=3)
plt.xlabel("Polynomial Degree")
plt.ylabel("Mean Squared Error")
plt.title("Comparison of MSE")
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.show()
```

```
# ====================================================
# SECTION 14: R-SQUARED COMPARISON GRAPH
# Study: Compare R2 scores of all models
# Higher R2 generally indicates better fit
# ====================================================

plt.figure(figsize=(8, 5))
plt.plot(
    results_df['Degree'],
    results_df['R-squared'],
    marker='o'
)
plt.xlabel("Polynomial Degree")
plt.ylabel("R-squared")
plt.title("Comparison of R-squared")
plt.grid(True)
plt.show()
```

```
# ====================================================
# EXPERIMENT 3
# LOGISTIC REGRESSION USING BREAST CANCER DATASET
# MLE vs MAP (L1 and L2)
# ====================================================



# ====================================================
# SECTION 1: IMPORT REQUIRED LIBRARIES
# Study: NumPy, Pandas, Matplotlib, sklearn
# Classification performance metrics
# ====================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    roc_curve,
    auc
)
```

```
# ======================================================
# SECTION 2: LOAD BREAST CANCER DATASET
# Study: X = input features, y = target labels
# Classes: 0 = Malignant, 1 = Benign
# ======================================================

data = load_breast_cancer()
X = data.data
y = data.target

print("Data Shape:", X.shape)
print("Classes:", np.unique(y))



# ======================================================
# SECTION 3: SPLIT TRAINING AND TESTING DATA
# Study: 80% training, 20% testing
# stratify=y preserves class proportions
# ======================================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
# ======================================================
# SECTION 4: FEATURE SCALING
# Study: StandardScaler standardizes features
# fit_transform for training data
# transform for testing data
# ======================================================

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)



# ======================================================
# SECTION 5: TRAIN MLE LOGISTIC REGRESSION MODEL
# Study: MLE = Maximum Likelihood Estimation
# penalty=None means no regularization
# Train model and predict test labels
# ======================================================

mle_model = LogisticRegression(
    penalty=None,
    max_iter=5000,
    random_state=42
)

mle_model.fit(X_train, y_train)
y_pred_mle = mle_model.predict(X_test)
```

```
# ====================================================
# SECTION 6: TRAIN MAP MODEL WITH L2 REGULARIZATION
# Study: L2 shrinks coefficients toward zero
# C controls regularization strength
# lbfgs solver supports L2 regularization
# ====================================================

map_l2 = LogisticRegression(
    penalty='l2',
    C=1.0,
    solver='lbfgs',
    max_iter=5000,
    random_state=42
)

map_l2.fit(X_train, y_train)
y_pred_l2 = map_l2.predict(X_test)



# ====================================================
# SECTION 7: TRAIN MAP MODEL WITH L1 REGULARIZATION
# Study: L1 can make some coefficients zero
# liblinear solver supports L1 regularization
# ====================================================

map_l1 = LogisticRegression(
    penalty='l1',
    C=1.0,
    solver='liblinear',
    max_iter=5000,
    random_state=42
```

```
)

map_l1.fit(X_train, y_train)
y_pred_l1 = map_l1.predict(X_test)



# ====================================================
# SECTION 8: DEFINE EVALUATION FUNCTION
# Study: Accuracy, Precision, Recall, F1 Score
# and Confusion Matrix
# ====================================================

def evaluate(model_name, y_true, y_pred):

    print("\n", "=" * 40)
    print(model_name)
    print("=" * 40)

    print("Accuracy:", accuracy_score(y_true, y_pred))
    print("Precision:", precision_score(y_true, y_pred))
    print("Recall  :", recall_score(y_true, y_pred))
    print("F1 Score:", f1_score(y_true, y_pred))

    print("\nConfusion Matrix")
    print(confusion_matrix(y_true, y_pred))
```

```
# ====================================================
# SECTION 9: EVALUATE ALL THREE MODELS
# Study: Compare MLE, MAP-L2 and MAP-L1
# ====================================================

evaluate("MLE", y_test, y_pred_mle)
evaluate("MAP (L2)", y_test, y_pred_l2)
evaluate("MAP (L1)", y_test, y_pred_l1)



# ====================================================
# SECTION 10: COMPARE ZERO COEFFICIENTS
# Study: Check how many feature coefficients
# become exactly zero in each model
# ====================================================

print("Number of Zero Coefficients")
print("MLE :", np.sum(mle_model.coef_[0] == 0))
print("MAP L2 :", np.sum(map_l2.coef_[0] == 0))
print("MAP L1 :", np.sum(map_l1.coef_[0] == 0))



# ====================================================
# SECTION 11: CREATE PERFORMANCE COMPARISON TABLE
# Study: Store Accuracy, Precision, Recall,
# F1 Score for all three models
# ====================================================

results = pd.DataFrame(columns=[
    "Model",
    "Accuracy",
```

```
    "Precision",
    "Recall",
    "F1 score"
])

models = [
    ("MLE", mle_model, y_pred_mle),
    ("MAP L2", map_l2, y_pred_l2),
    ("MAP L1", map_l1, y_pred_l1)
]

for name, model, pred in models:
    results.loc[len(results)] = [
        name,
        accuracy_score(y_test, pred),
        precision_score(y_test, pred),
        recall_score(y_test, pred),
        f1_score(y_test, pred)
    ]

print(results)



# ==================================================
# SECTION 12: PERFORMANCE COMPARISON BAR GRAPH
# Study: Plot Accuracy, Precision, Recall and F1
# Compare the three Logistic Regression models
# ==================================================

ax = results.set_index("Model").plot(
    kind="bar",
```

```
    figsize=(10, 6)
)

plt.title("Performance Comparison: MLE vs MAP-L2 vs MAP-L1")
plt.xlabel("Model")
plt.ylabel("Score")

plt.ylim(0.85, 1.02)
plt.xticks(rotation=0)
plt.legend(title="Metric")
plt.grid(axis="y", alpha=0.3)

for container in ax.containers:
    ax.bar_label(container, fmt="%.3f", padding=2)

plt.tight_layout()
plt.show()



# ======================================================
# SECTION 13: ROC CURVE AND AUC CALCULATION
# Study: predict_proba() gives probabilities
# roc_curve() calculates FPR and TPR
# auc() calculates area under ROC curve
# ======================================================

plt.figure(figsize=(9, 7))

for name, model, pred in models:
    y_prob = model.predict_proba(X_test)[:, 1]
    fpr, tpr, thresholds = roc_curve(y_test, y_prob)
```

```
    roc_auc = auc(fpr, tpr)

    plt.plot(
        fpr,
        tpr,
        label=f"{name} (AUC={roc_auc:.3f})"
    )


# ====================================================
# SECTION 14: ROC GRAPH FORMATTING
# Study: Plot random classifier reference line
# Add labels, title, legend and grid
# ====================================================

plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random Classifier"
)

plt.xlim(0, 0.15)
plt.ylim(0.75, 1.01)
plt.xlabel("False Positive Rate (FPR)")
plt.ylabel("True Positive Rate (TPR)")
plt.title("ROC curve: MLE vs MAP L2 vs MAP L1")
plt.legend(loc="lower right")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
# ====================================================
# EXPERIMENT 4
# NAIVE BAYES CLASSIFICATION USING
# THE 20 NEWSGROUPS DATASET
# ====================================================



# ====================================================
# SECTION 1: IMPORT REQUIRED LIBRARIES
# Study: NumPy, Pandas, Matplotlib, sklearn
# CountVectorizer and Naive Bayes classifiers
# ====================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)
```

```
# ====================================================
# SECTION 2: LOAD TRAINING AND TESTING DATASETS
# Study: Load 20 Newsgroups dataset
# Remove headers, footers and quotes
# ====================================================

train_data = fetch_20newsgroups(
    subset="train",
    remove=("headers", "footers", "quotes")
)

test_data = fetch_20newsgroups(
    subset="test",
    remove=("headers", "footers", "quotes")
)



# ====================================================
# SECTION 3: SEPARATE TEXT AND TARGET LABELS
# Study: Extract training/testing documents
# and their corresponding class labels
# ====================================================

X_train_text = train_data.data
y_train = train_data.target

X_test_text = test_data.data
y_test = test_data.target

class_names = train_data.target_names
```

```
print("Training documents:", len(X_train_text))
print("Testing documents:", len(X_test_text))
print("Number of classes:", len(class_names))


# ==================================================
# SECTION 4: TEXT VECTORIZATION
# Study: Convert text into numerical word counts
# Remove English stop words
# Ignore words appearing in fewer than 2 documents
# ==================================================

vectorizer = CountVectorizer(
    stop_words="english",
    min_df=2
)

X_train = vectorizer.fit_transform(X_train_text)
X_test = vectorizer.transform(X_test_text)


# ==================================================
# SECTION 5: DISPLAY FEATURE MATRIX SHAPES
# Study: Check number of documents and features
# ==================================================

print("Traing feature matrix:", X_train.shape)
print("Testing feature matrix:", X_test.shape)
```

```
# ======================================================
# SECTION 6: MULTINOMIAL NAIVE BAYES
# Study: Train MultinomialNB using word counts
# Predict classes for test documents
# ======================================================

mnb = MultinomialNB()

mnb.fit(
    X_train,
    y_train
)

y_pred_mnb = mnb.predict(X_test)



# ======================================================
# SECTION 7: BINARY TEXT VECTORIZATION
# Study: Convert text into binary features
# 1 = word present, 0 = word absent
# Used for Bernoulli Naive Bayes
# ======================================================

binary_vectorizer = CountVectorizer(
    stop_words="english",
    min_df=2,
    binary=True
)

X_train_binary = binary_vectorizer.fit_transform(X_train_text)
X_test_binary = binary_vectorizer.transform(X_test_text)
```

```
# ====================================================
# SECTION 8: BERNOULLI NAIVE BAYES
# Study: Train BernoulliNB using binary features
# Predict classes for test documents
# ====================================================

bnb = BernoulliNB()

bnb.fit(
    X_train_binary,
    y_train
)

y_pred_bnb = bnb.predict(X_test_binary)



# ====================================================
# SECTION 9: EVALUATE BOTH CLASSIFIERS
# Study: Calculate Accuracy, Precision,
# Recall and F1-Score
# macro = equal importance to all classes
# ====================================================

results = pd.DataFrame({
    "Model": [
        "Multinomial NB",
        "Bernoulli NB"
    ],
    "Accuracy": [
```

```
    accuracy_score(y_test, y_pred_mnb),
    accuracy_score(y_test, y_pred_bnb)
],
"Precision": [
    precision_score(
        y_test,
        y_pred_mnb,
        average="macro"
    ),

    precision_score(
        y_test,
        y_pred_bnb,
        average="macro"
    )
],
"Recall": [
    recall_score(
        y_test,
        y_pred_mnb,
        average="macro"
    ),

    recall_score(
        y_test,
        y_pred_bnb,
        average="macro"
    )
],
"F1-Score": [
    f1_score(
```

```
            y_test,
            y_pred_mnb,
            average="macro"
        ),

        f1_score(
            y_test,
            y_pred_bnb,
            average="macro"
        )
    ]
})

print(results)



# ======================================================
# SECTION 10: PREPARE RESULTS FOR COMPARISON
# Study: Select metrics from results DataFrame
# ======================================================

results_metrics = results[
    ["Model", "Accuracy", "Precision", "Recall", "F1-Score"]
]
```

```
# ====================================================
# SECTION 11: PERFORMANCE COMPARISON BAR GRAPH
# Study: Compare MultinomialNB and BernoulliNB
# using Accuracy, Precision, Recall and F1
# ====================================================

ax = results_metrics.set_index("Model").plot(
    kind="bar",
    figsize=(11, 6)
)

plt.title("Performance Comparison: Multinomial NB vs Bernoulli NB")
plt.xlabel("Model")
plt.ylabel("Score")
plt.ylim(0, 1.02)
plt.xticks(rotation=0)
plt.legend(title="Metric")
plt.grid(
    axis="y",
    alpha=0.3
)
```

```
# ====================================================
# SECTION 12: DISPLAY VALUES AND SHOW GRAPH
# Study: Add metric values above bars
# Adjust layout and display graph
# ====================================================

for container in ax.containers:
    ax.bar_label(
        container,
        fmt="%.3f",
        padding=2
    )

plt.tight_layout()
plt.show()
```

```
# ====================================================
# EXPERIMENT 5
# IMAGE CLASSIFICATION USING K-NEAREST NEIGHBORS
# (KNN) ON FASHION-MNIST DATASET
# ====================================================



# ====================================================
# SECTION 1: IMPORT REQUIRED LIBRARIES
# Study: NumPy, time, sklearn and warnings
# KNeighborsClassifier is used for KNN
# ====================================================

import numpy as np
import time
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import warnings

warnings.filterwarnings('ignore', category=FutureWarning)
```

```
# ======================================================
# SECTION 2: LOAD FASHION-MNIST DATASET
# Study: Download dataset using fetch_openml()
# X = image pixels, y = clothing labels
# ======================================================

print("Downloading Fashion MNIST dataset (this may take 30-60 seconds)....")

fashion_mnist = fetch_openml('Fashion-MNIST', version=1, as_frame=False)

X = fashion_mnist.data
y = fashion_mnist.target.astype(int)

print(f"Total dataset size: {X.shape[0]} images,each with {X.shape[1]} pixels.")



# ======================================================
# SECTION 3: CREATE STRATIFIED SUBSET
# Study: Select 12000 images from the dataset
# stratify=y maintains class proportions
# ======================================================

X_subset, _, y_subset, _ = train_test_split(
    X, y,
    train_size=12000,
    stratify=y,
    random_state=42
)
```

```
# ======================================================
# SECTION 4: SPLIT TRAINING AND TESTING DATA
# Study: 10000 training images
# 2000 testing images
# ======================================================

X_train, X_test, y_train, y_test = train_test_split(
    X_subset, y_subset,
    test_size=2000,
    stratify=y_subset,
    random_state=42
)

print(f"Training images: {X_train.shape[0]}")
print(f"Testing images: {X_test.shape[0]}")



# ======================================================
# SECTION 5: FEATURE SCALING
# Study: Normalize pixel values from 0-255 to 0-1
# Divide pixel values by 255.0
# ======================================================

print(f"Before scaling -> Min: {X_train.min()},Max: {X_train.max()}")

X_train = X_train / 255.0
X_test = X_test / 255.0

print(f"After scaling -> Min: {X_train.min()},Max:{X_train.max()}")
```

```
# ======================================================
# SECTION 6: INITIALIZE K VALUES AND RESULTS
# Study: Test different numbers of neighbors
# Store accuracy, prediction time and predictions
# ======================================================

k_values = [1, 3, 5, 7, 9, 15]
results = {}

print(f"{'K values':<8} | {'Accuracy':<10} | {'Prediction Time (seconds)':<25}")
print("-" * 50)



# ======================================================
# SECTION 7: TRAIN KNN FOR DIFFERENT K VALUES
# Study: Euclidean distance, fit(), predict()
# Calculate prediction time and accuracy
# ======================================================

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k, metric='euclidean', n_jobs=1)


    knn.fit(X_train, y_train)


    start_time = time.time()


    y_pred = knn.predict(X_test)


    elapsed_time = time.time() - start_time


    acc = accuracy_score(y_test, y_pred)
```

```
    results[k] = {
        "accuracy": acc,
        "time": elapsed_time,
        "prediction": y_pred
    }

    print(f"{k:<8} | {acc * 100:<9.2f}% | {elapsed_time:<25.2f}")


# ======================================================
# SECTION 8: DEFINE FASHION-MNIST CLASS NAMES
# Study: 10 clothing categories
# Labels range from 0 to 9
# ======================================================

class_names = [
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"
]


# ======================================================
# SECTION 9: FIND THE BEST K VALUE
# Study: Select K with highest classification accuracy
# ======================================================

best_k = max(results, key=lambda k: results[k]["accuracy"])

print(f"Best K is: {best_k} with {results[best_k]['accuracy'] * 100:.2f} % accuracy\n")
```

```
# ====================================================
# SECTION 10: DISPLAY CLASSIFICATION REPORT
# Study: Precision, Recall, F1-score and Support
# for each of the 10 clothing classes
# ====================================================

print("Per-Class Classification Report:")

print(classification_report(
    y_test,
    results[best_k]["prediction"],
    target_names=class_names
))
```

```
# ====================================================
# EXPERIMENT 6
# CUSTOMER SEGMENTATION USING
# AN ID3-STYLE DECISION TREE
# ====================================================


# ====================================================
# SECTION 1: IMPORT REQUIRED LIBRARIES
# Study: Pandas, NumPy, Matplotlib and sklearn
# DecisionTreeClassifier and plot_tree
# ====================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.tree import DecisionTreeClassifier, plot_tree


# ====================================================
# SECTION 2: LOAD ONLINE RETAIL DATASET
# Study: Read Excel file using pd.read_excel()
# Display first five rows and dataset information
# ====================================================

df = pd.read_excel(r"C:\Users\CSEC2\Downloads\Online Retail.xlsx")
```

```
print(df.head())
print(df.info())



# ====================================================
# SECTION 3: DATA CLEANING
# Study: Remove missing CustomerID values
# Remove cancelled invoices starting with C
# Keep only positive Quantity and UnitPrice
# Calculate TotalAmount
# ====================================================

df = df.dropna(subset=["CustomerID"])
df = df[~df["InvoiceNo"].astype(str).str.startswith("C")]
df = df[df["Quantity"] > 0]
df = df[df["UnitPrice"] > 0]

df["TotalAmount"] = df["Quantity"] * df["UnitPrice"]

print(df.shape)
print(df.head())
```

```
# ====================================================
# SECTION 4: CUSTOMER-LEVEL FEATURE AGGREGATION
# Study: Group transactions by CustomerID
# Calculate spending, quantity, transactions,
# products, average order value and country
# ====================================================

customer_data = df.groupby("CustomerID").agg(
    TotalSpend=("TotalAmount", "sum"),
    TotalQuantity=("Quantity", "sum"),
    NumberOfTransactions=("InvoiceNo", "nunique"),
    NumberOfProducts=("StockCode", "nunique"),
    AverageOrderValue=("TotalAmount", "mean"),
    Country=("Country", "first")
).reset_index()

print(customer_data.head())

# ====================================================
# SECTION 5: CREATE CUSTOMER SEGMENTS
# Study: pd.qcut() divides customers into
# three groups based on TotalSpend
# Low Value, Medium Value and High Value
# ====================================================

customer_data["Segment"] = pd.qcut(
    customer_data["TotalSpend"],
    q=3,
    labels=["Low Value", "Medium Value", "High Value"]
)
print(customer_data["Segment"].value_counts())
```

```
# ====================================================
# SECTION 6: SELECT INPUT FEATURES AND TARGET
# Study: X = customer behavior features
# y = customer segment
# TotalSpend is excluded from X
# ====================================================

features = [
    "TotalQuantity",
    "NumberOfTransactions",
    "NumberOfProducts",
    "AverageOrderValue"
]

X = customer_data[features]
y = customer_data["Segment"]



# ====================================================
# SECTION 7: SPLIT TRAINING AND TESTING DATA
# Study: 80% training and 20% testing
# stratify=y preserves segment proportions
# ====================================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

```
# ====================================================
# SECTION 8: TRAIN ID3-STYLE DECISION TREE
# Study: entropy is used for information gain
# max_depth=4 limits tree depth
# fit() trains the classifier
# ====================================================

id3_model = DecisionTreeClassifier(
    criterion="entropy",
    random_state=42,
    max_depth=4
)

id3_model.fit(X_train, y_train)



# ====================================================
# SECTION 9: PREDICT CUSTOMER SEGMENTS
# Study: predict() classifies test customers
# Display the first 20 predictions
# ====================================================

y_pred = id3_model.predict(X_test)

print("Prediction")
print(y_pred[:20])
```

```
# ====================================================
# SECTION 10: EVALUATE DECISION TREE MODEL
# Study: Accuracy, Classification Report
# and Confusion Matrix
# ====================================================

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# ====================================================
# SECTION 11: CALCULATE FEATURE IMPORTANCE
# Study: Find which customer features contribute
# most to decision tree predictions
# ====================================================

importance = pd.DataFrame({
    "Feature": features,
    "Importance": id3_model.feature_importances_
})

importance_sorted = importance.sort_values(
    "Importance",
    ascending=True
)
```

```
# ====================================================
# SECTION 12: VISUALIZE THE DECISION TREE
# Study: plot_tree() draws the trained tree
# Shows feature splits and customer classes
# ====================================================

plt.figure(figsize=(22, 12))

plot_tree(
    id3_model,
    feature_names=features,
    class_names=id3_model.classes_,
    filled=True,
    rounded=True,
    proportion=False
)

plt.title("Customer Segmentation using ID3 Decision Tree")
plt.show()



# ====================================================
# SECTION 13: PLOT FEATURE IMPORTANCE GRAPH
# Study: Horizontal bar chart shows the
# importance of each customer behavior feature
# ====================================================

plt.figure(figsize=(10, 6))

importance_sorted = importance.sort_values(
    "Importance",
```

```
    ascending=True
)

plt.barh(
    importance_sorted["Feature"],
    importance_sorted["Importance"]
)

plt.xlabel("Information_based importance")
plt.ylabel("Customer Behavior Feature")
plt.title("Feature Importance in Customer Segmentation")
plt.show()
```

```
# ====================================================
# EXPERIMENT 7
# SUPPORT VECTOR MACHINE (SVM) CLASSIFICATION
# USING FASHION-MNIST DATASET
# ====================================================



# ====================================================
# SECTION 1: IMPORT REQUIRED LIBRARIES
# Study: NumPy, Pandas, Matplotlib and sklearn
# SVC is used for Support Vector Classification
# ====================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report



# ====================================================
# SECTION 2: LOAD FASHION-MNIST DATASET
# Study: fetch_openml() loads the dataset
# X = image pixels, y = clothing class labels
# Convert pixels to float32 and labels to integer
# ====================================================
```

```
print("Loading Fashion MNIST dataset.....")

X, y = fetch_openml(
    'Fashion-MNIST',
    version=1,
    return_X_y=True,
    as_frame=False
)

X = X.astype('float32')
y = y.astype('int')

print("Dataset shape", X.shape)



# ======================================================
# SECTION 3: SELECT FIRST 10000 SAMPLES
# Study: Reduce dataset size for faster training
# ======================================================

X = X[:10000]
y = y[:10000]



# ======================================================
# SECTION 4: SPLIT TRAINING AND TESTING DATA
# Study: 80% training (8000 samples)
# 20% testing (2000 samples)
# stratify=y preserves class proportions
# ======================================================
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training samples:", X_train.shape[0])
print("Testing sample:", X_test.shape[0])



# ====================================================
# SECTION 5: FEATURE SCALING
# Study: StandardScaler standardizes pixel features
# fit_transform() on training data
# transform() on testing data
# ====================================================

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)



# ====================================================
# SECTION 6: TRAIN LINEAR SVM MODEL
# Study: SVC(kernel='linear') uses linear kernel
# fit() trains the classifier
# predict() predicts clothing classes
# ====================================================
```

```
print("\nTraining Linear SVM...")

linear_svm = SVC(kernel='linear')

linear_svm.fit(X_train, y_train)

y_pred_linear = linear_svm.predict(X_test)

linear_accuracy = accuracy_score(y_test, y_pred_linear)

print("Linear SVM Accuracy:", linear_accuracy * 100, "%")



# ====================================================
# SECTION 7: TRAIN RBF SVM MODEL
# Study: RBF = Radial Basis Function
# RBF kernel handles nonlinear relationships
# gamma='scale' automatically calculates gamma
# ====================================================

print("\nTraining RBF SVM...")

rbf_svm = SVC(
    kernel='rbf',
    gamma='scale'
)

rbf_svm.fit(X_train, y_train)

y_pred_rbf = rbf_svm.predict(X_test)
```

```
rbf_accuracy = accuracy_score(y_test, y_pred_rbf)

print("RBF SVM Accuracy:", rbf_accuracy * 100, "%")


# ==================================================
# SECTION 8: DISPLAY CLASSIFICATION REPORTS
# Study: Compare Precision, Recall, F1-score
# and Support for Linear and RBF SVM
# ==================================================

print("\n---Linear SVM Classifion Report---")
print(classification_report(y_test, y_pred_linear))

print("\n---RBF SVM Classification Report---")
print(classification_report(y_test, y_pred_rbf))


# ==================================================
# SECTION 9: PREPARE ACCURACY COMPARISON
# Study: Store model names and accuracies
# for plotting the bar graph
# ==================================================

models = ['Linear SVM', 'RBF SVM']
accuracy = [linear_accuracy * 100, rbf_accuracy * 100]
```

```
# ====================================================
# SECTION 10: CREATE ACCURACY COMPARISON BAR GRAPH
# Study: Compare Linear SVM and RBF SVM accuracy
# ====================================================

plt.figure(figsize=(7, 5))

bars = plt.bar(models, accuracy)

plt.ylabel("Accuracy (%)")
plt.xlabel("SVM Kernel")
plt.title("Performance Comparison of SVM Kernels")
plt.ylim(0, 100)



# ====================================================
# SECTION 11: DISPLAY ACCURACY VALUES ON BARS
# Study: Add percentage labels above bars
# Show the comparison graph
# ====================================================

for bar, value in zip(bars, accuracy):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value + 1,
        f"{value:.2f}%",
        ha='center'
    )

plt.show()
```